Matrix Factorization & Two Tower

Dr Seungmin Jin
sedzhin@hse.ru

HSE

Course Objectives

- Explain why sparse ratings can make similarity-based recommendations unreliable.
- Understand how matrix factorization represents users and items as embeddings.
- Calculate dot products to score user–item matches.
- Distinguish recommendation-oriented matrix factorization from classical SVD.
- Explain how two-tower models generate embeddings for scalable candidate retrieval.

When Similarity Is Built on Too Little Evidence

Sparsity: two users may share only a few ratings.

Sparse rating matrix

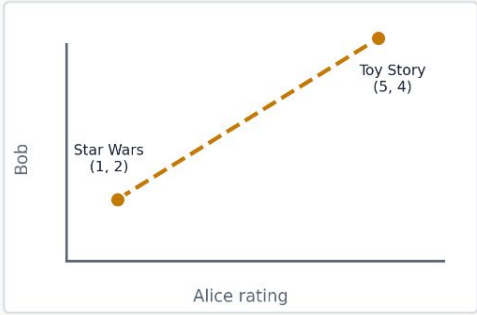
Alice and Bob share only 2 ratings

	Toy Story	Star Wars	Dune	Interstellar	Spirited Away	Parasite
Alice	5	1	?	?	?	?
Bob	4	2	?	?	?	?
Carol	?	?	5	4	3	?
Dave	?	?	4	5	?	2

The pale cells are the complete evidence used to calculate sim(Alice, Bob).

What does Pearson see?

Two observations: (5, 4) and (1, 2).



Pearson r = +1.00

Perfect correlation — from only two points.

The problem is not whether a similarity score can be computed; it is whether the estimate is trustworthy.

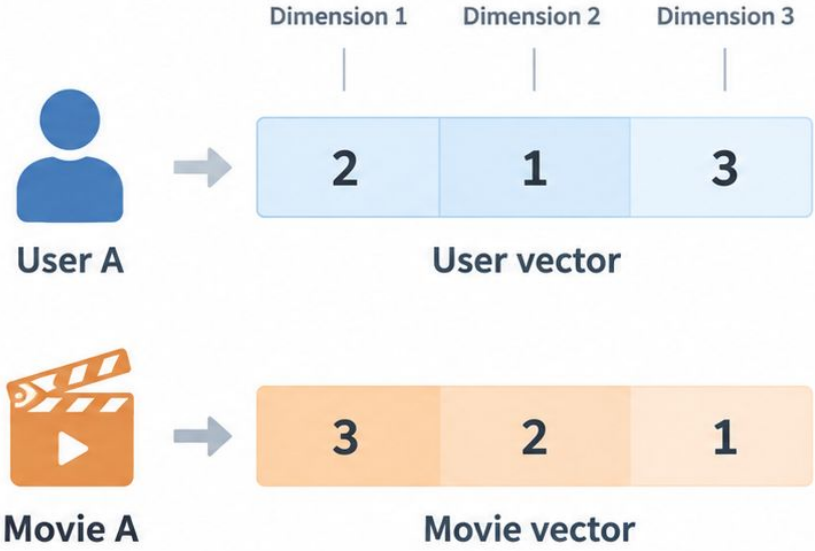
2 co-rated items → Similarity is computable → Estimate is fragile → Unreliable neighbor

Represent Users and Movies as Vectors

Observed Ratings

	Movie A	Movie B	Movie C
User A	5	—	2
User B	—	4	—
User C	3	—	5

Compact representations



Each user and movie is represented by a short list of numbers.

Illustrative vectors. Values are learned from data; dimensions need not have explicit meanings.

Dot Product: Two Vectors, One Score

P: Users — Q^T: Movies — $\hat{R}$: Predicted scores

User B

$$\begin{bmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 2 \end{bmatrix} \times \begin{bmatrix} 1 & 3 & 2 \\ 2 & 4 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 3 & 2 \\ 4 & \square & 5 \\ 4 & 8 & 2 \end{bmatrix}$$

Movie B

$$\hat{r}_{B,B} = \begin{bmatrix} 2 & 1 \end{bmatrix} \times \begin{bmatrix} 3 \\ 4 \end{bmatrix}$$

$$= 2 \times 3 + 1 \times 4 = 6 + 4 = \mathbf{10}$$

One user row × one movie column = one predicted entry.

Illustrative values; bias terms omitted.

Learning from Observed Ratings

1 Predict

$$\begin{bmatrix} 2 & 1 \end{bmatrix} \times \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \mathbf{10}$$

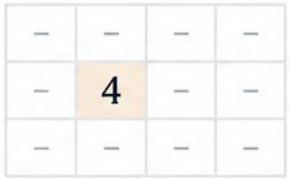
User vector

Movie vector

Current prediction

2 Measure error

Observed rating: 4



$$L = (4 - 10)^2 = \mathbf{36}$$

3 Adjust vectors

Illustrative update

$$\begin{bmatrix} 1.5 & 0.5 \end{bmatrix} \times \begin{bmatrix} 2.5 \\ 3.5 \end{bmatrix} = \mathbf{5.5}$$

$$L_{new} = (4 - 5.5)^2 = \mathbf{2.25}$$

Predict → **Compare with observed rating** → **Adjust** → **Repeat**

Learn user and movie vectors that reduce prediction error.

Illustrative values; not an exact gradient step. Training uses many observed ratings. Bias terms, regularization, and derivatives are omitted.

MF and SVD: Related, Not Identical

Classical SVD

$$A = U\Sigma V^{\mathrm{T}}$$

$$A = \begin{bmatrix} 5 & 3 & 1 \\ 4 & 2 & 1 \\ 1 & 2 & 5 \end{bmatrix}$$

A fully specified matrix

- Exact decomposition of a numerical matrix
- Truncate singular values for a low-rank approximation

$$A \approx U_k \Sigma_k V_k^{\mathrm{T}}$$

Rating-based Matrix Factorization

$$R \approx PQ^{\mathrm{T}}$$

$$R = \begin{bmatrix} 5 & - & 1 \\ - & 4 & - \\ 2 & - & 5 \end{bmatrix}$$

Observed ratings with missing entries

- Learn user and item vectors
- Fit observed ratings—not missing entries

$$\min_{P,Q} \sum_{(u,i)\in\Omega} (r_{ui} - \boldsymbol{p}_u^{\mathrm{T}} \boldsymbol{q}_i)^2$$

Ω = observed user–item pairs; regularization omitted

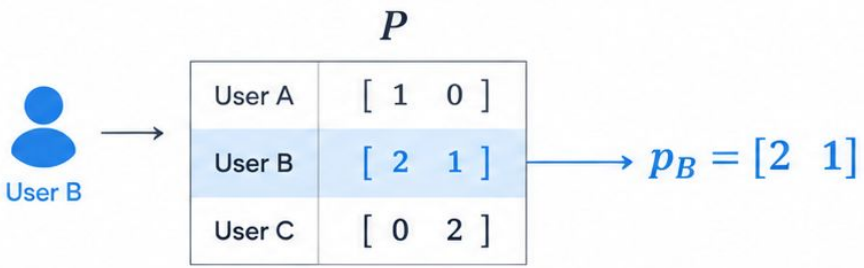
Shared idea: low-rank representations. Different treatment of missing data.

Missing rating $\neq$ zero rating.

Explicit-rating MF shown. Classical SVD requires missing values to be handled separately.

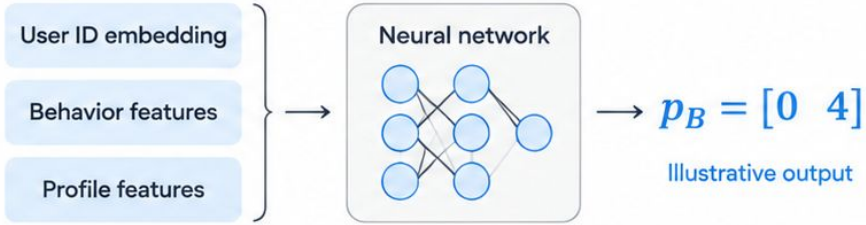
From Vector Lookup to Vector Generation

ID-based MF: Look up a vector



One learned vector per user ID

Feature-based tower: Generate a vector



$$p_B = f_\theta(x_B)$$

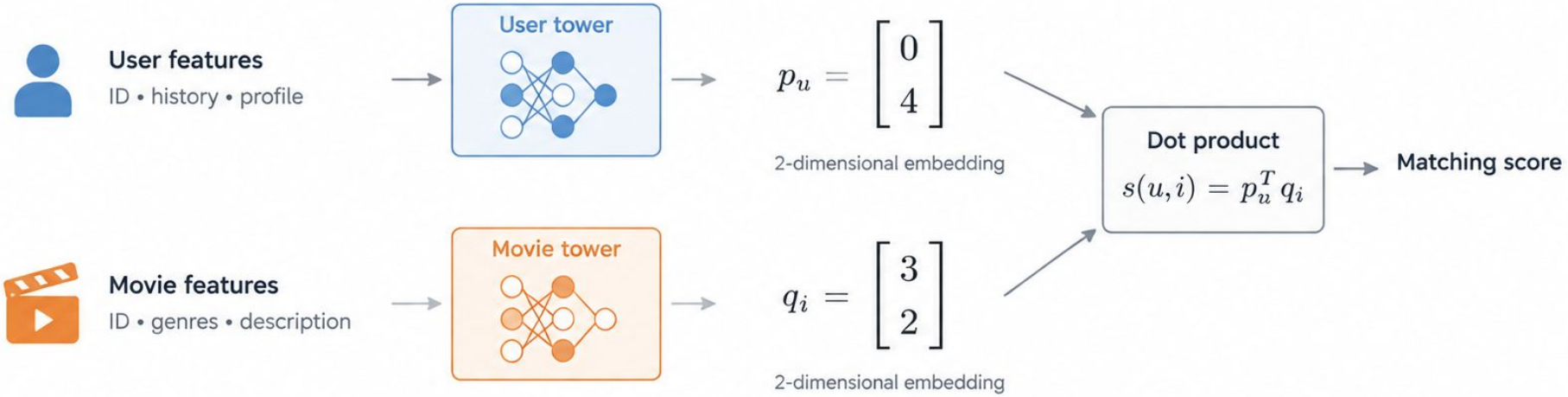
Learn a function that maps features to a vector

Learn vectors directly → Learn how to produce vectors

Both produce an embedding. The way it is obtained changes.

Illustrative values. Towers may include ID embeddings; feature-based models still learn parameters.

Two Towers, Two Embeddings

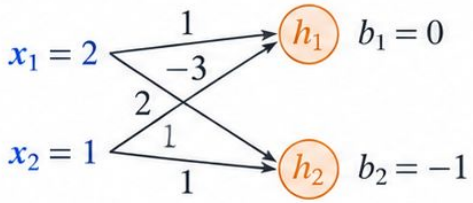


$$s(u,i) = \begin{bmatrix} 0 & 4 \end{bmatrix} \begin{bmatrix} 3 \\ 2 \end{bmatrix} = 0 \times 3 + 4 \times 2 = 8$$

Separate encoders. Same embedding dimension. One matching score.

Illustrative outputs; 2 dimensions shown for clarity. Training loss is not shown.

A Neural Network: A Numerical Example

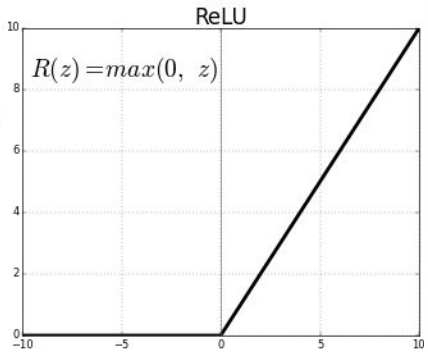


Neuron 1: $z_1 = 1 \times 2 + (-3) \times 1 + 0 = -1$

Neuron 2: $z_2 = 2 \times 2 + 1 \times 1 - 1 = 4$

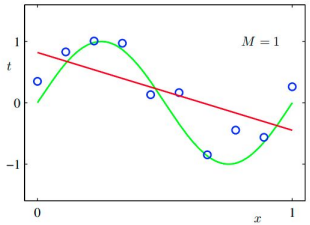
$\mathrm{ReLU}(z) = \max(0, z)$

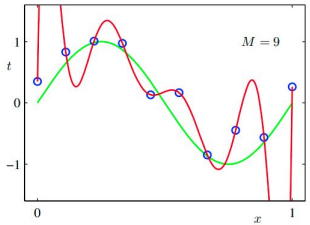
$h_1 = 0, h_2 = 4$



$$z = Wx + b = \begin{bmatrix} 1 & -3 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \end{bmatrix} \xrightarrow{\text{ReLU}} \begin{bmatrix} -1 \\ 4 \end{bmatrix}$$

$$h = \mathrm{ReLU}(z) = \begin{bmatrix} 0 \\ 4 \end{bmatrix}$$



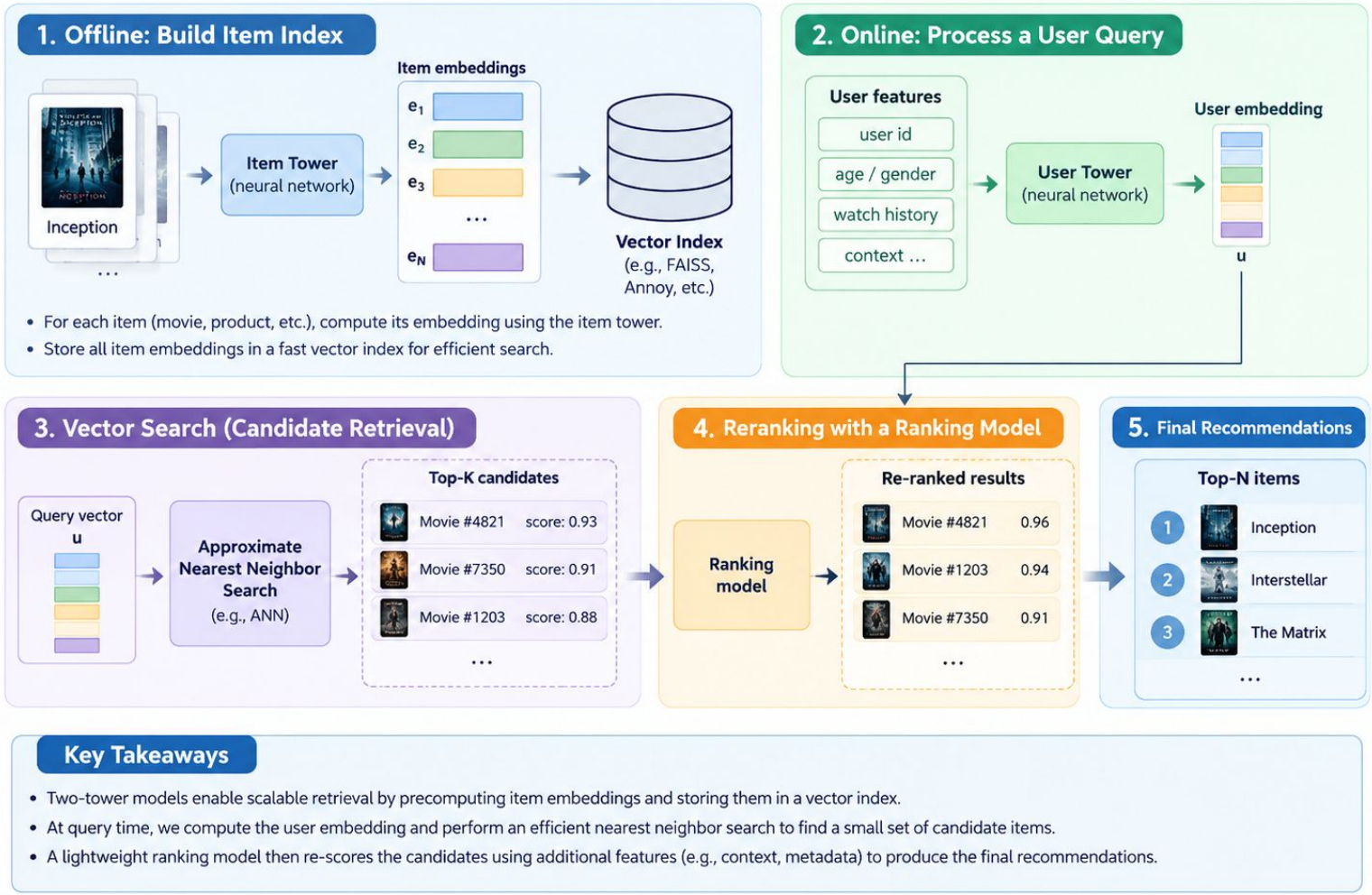


Each neuron produces one number. Together, they form an output vector.

$$h = \mathrm{ReLU}(Wx + b)$$

Illustrative weights and biases. Training adjusts these parameters; gradients are omitted.

Retrieve Candidates at Scale



Key Takeaways

- Two-tower models enable scalable retrieval by precomputing item embeddings and storing them in a vector index.
- At query time, we compute the user embedding and perform an efficient nearest neighbor search to find a small set of candidate items.
- A lightweight ranking model then re-scores the candidates using additional features (e.g., context, metadata) to produce the final recommendations.

Precision@K, Recall@K and NDCG@K

Count the hits. Check coverage. Reward better ordering.

Relevant test movies	A	B	C	D	
Top-5 recommendations	A	E	B	F	C

3 hits at ranks 1, 3 and 5 · 5 recommendations · 4 relevant test movies

Precision@K

How much of the list is relevant?

$$P@K = \frac{\text{Hits in Top-}K}{K}$$

$$P@5 = \frac{3}{5} = 60\%$$

Denominator: list size.
Does not distinguish hit positions.

Recall@K

How much of the relevant set did we find?

$$R@K = \frac{\text{Hits in Top-}K}{\text{Relevant test items}}$$

$$R@5 = \frac{3}{4} = 75\%$$

Denominator: relevant set size.
Movie D was missed.

NDCG@K

Are relevant items near the top?

$$\text{DCG@}K = \sum_{r=1}^{K} \frac{\text{rel}_r}{\log_2(r+1)}$$

Rank	1	2	3	4	5
Relevance	1	0	1	0	1
Discount	1.000	0.631	0.500	0.431	0.387

DCG@5 = 1 + 0.500 + 0.387 ≈ **1.887**
IDCG@5 = 1 + 0.631 + 0.500 + 0.431 ≈ **2.562**

Ideal top-5: A, B, C, D, E.
Use all 4 relevant items—not just the 3 hits.

$$\text{NDCG@5} = \frac{1.887}{2.562} \approx 0.737$$

Reorder the same five items to **A, B, C, E, F**: precision stays 60%; recall stays 75%; NDCG rises to **0.832**

DCG and IDCG: Actual Score vs. Ideal Score

Relevant items earn credit. Earlier relevant items earn more credit.

Relevant test movies: **A, B, C, D**

Actual top-5 recommendations: **A, E, B, F, C**

DCG: Actual ranking score

Rank	Movie	Relevant?	Credit
1	A	Yes	1.000
2	E	No	0
3	B	Yes	0.500
4	F	No	0
5	C	Yes	0.387

DCG@5 = 1 + 0.500 + 0.387
= **1.887**

IDCG: Best possible score

Put all four relevant movies first.

Ideal ranking: A → B → C → D → E

IDCG@5 = 1 + 0.631 + 0.500 + 0.431
= **2.562**

Include D, even though the actual ranking missed it. E is a non-relevant filler; F gives the same score.

Credit at rank r = relevance / $\log_2(r + 1)$. Relevant = 1; otherwise = 0.
DCG = Discounted Cumulative Gain · IDCG = Ideal DCG.

NDCG: How Close Are We to the Ideal?

Normalize the actual score by the best possible score for the same user.

$$\text{NDCG@5} = \text{DCG@5} / \text{IDCG@5}$$
$$1.887 / 2.562 \approx \mathbf{0.737}$$

DCG = earned points

Rewards relevant items and discounts lower positions.

IDCG = maximum points

Uses all relevant candidates, not only the actual hits.

NDCG = earned / maximum

0 to 1 when IDCG > 0. An ideal ranking scores 1.

Same hits, better order → higher NDCG

Top-5 ranking	Hits	Precision@5	Recall@5	NDCG@5
A, E, B, F, C	3	60%	75%	0.737
A, B, C, E, F	3	60%	75%	0.832
A, B, C, D, E	4	80%	100%	1.000

Row 2 reorders the same items. Row 3 replaces F with the missed relevant movie D.

[Normalized Discounted Cumulative Gain (NDCG) explained]

> 0.737 means 73.7% of the ideal weighted score—not purchase probability.
> Precision counts hits; recall measures coverage; NDCG also rewards better order.

MAP@K — Mean Average Precision at K

How well do we place relevant items near the top?

1. Precision at rank r

$$P_u@r = \frac{\sum_{j=1}^{r} rel_u(j)}{r}$$

Look at the first r recommendations.

What proportion are relevant?

2. Average Precision: one user

$$AP_u@K = \frac{\sum_{r=1}^{K} (P_u@r) rel_u(r)}{\min(K, |R_u|)}$$

Scan the list from top to bottom.
At each relevant item, add precision so far.
Normalize by min(K, total relevant items).

Irrelevant items add no points, but lower precision at later relevant ranks.

3. Mean Average Precision: all users

$$\text{MAP@}K = \frac{1}{|U|} \sum_{u \in U} AP_u@K$$

Compute AP@K for each user.

Average those scores for the same method.

$rel_u(r)$: 1 if relevant, 0 otherwise · R_u: ground-truth relevant items · U: evaluated users

Precision: relevance proportion → AP: one user's ranking → MAP: average across users

Convention shown: $\min(K, |R_u|)$ normalization. Match your evaluator's definition.

MAP@5: Correct recommendations, earlier

Two lists can find the same programs — but rank them differently.

One user · 5 recommendations · **Relevant programs: A, B, C**

Method X

A → B → D → C → E

At each correct recommendation, check precision so far:

Rank 1: 1 correct out of 1 → 1.00

Rank 2: 2 correct out of 2 → 1.00

Rank 4: 3 correct out of 4 → 0.75

AP@5 = (1.00 + 1.00 + 0.75) / 3 ≈ 0.917

Method Y

D → A → E → B → C

At each correct recommendation, check precision so far:

Rank 2: 1 correct out of 2 → 0.50

Rank 4: 2 correct out of 4 → 0.50

Rank 5: 3 correct out of 5 → 0.60

AP@5 = (0.50 + 0.50 + 0.60) / 3 ≈ 0.533

Both lists: Precision@5 = 3/5 = 0.60 · Recall@5 = 3/3 = 1.00

AP scores one user: earlier correct items earn a higher score.
MAP averages AP across users — separately for each method.

AP@K convention: sum precision at relevant ranks, then divide by min(K, number of ground-truth relevant items).
Here the denominator is min(5, 3) = 3, not the number of hits. Match your evaluator's convention.

Recommender systems at a glance

1) Fill the blanks using people with similar taste 2) Narrow millions of items down to a few

1. Collaborative filtering: similar taste, similar liking

Similar taste

Min and I gave the same ratings on the first two movies

	Avengers	Inception	La La Land
Min	5	4	5
Me	5	4	?

borrow Min's 5

Prediction: La La Land ≈ 5

Min liked it, so I probably will too, so it joins my list

Prediction accuracy is measured with RMSE (lower is better)

Find neighbors with similar taste, and you can fill in what I have not watched yet.

2. Real systems use a funnel: a huge pool becomes a short list

All items

100,000 movies (orange = kept)



Quickly narrow

Candidate retrieval

Keep about 500 candidates

A B C D E

... and about 495 more

Recall@K: did we keep the relevant ones?

Rank carefully

Final ranking

Show only the Top-10

1 Avengers

2 Inception

3 La La Land

NDCG@K: are good items near the top?

Takeaway: fill the blanks with similar users, then narrow a huge pool to the Top-K

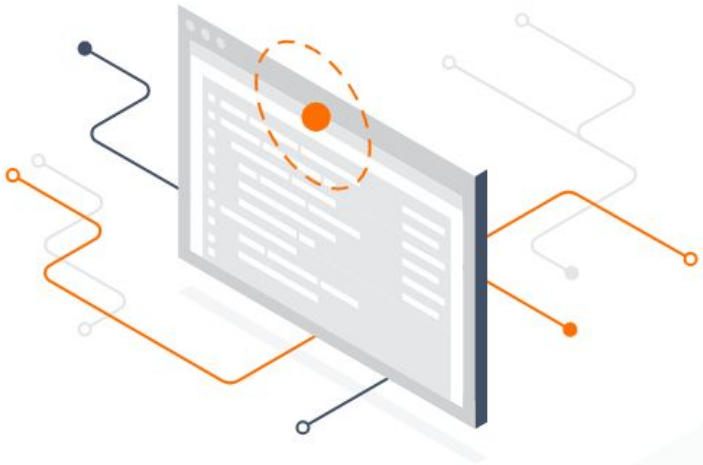
Discussion

TensorFlow.js

TensorFlow.js is a library for machine learning in JavaScript

Develop ML models in JavaScript, and use ML directly in the browser or in Node.js.

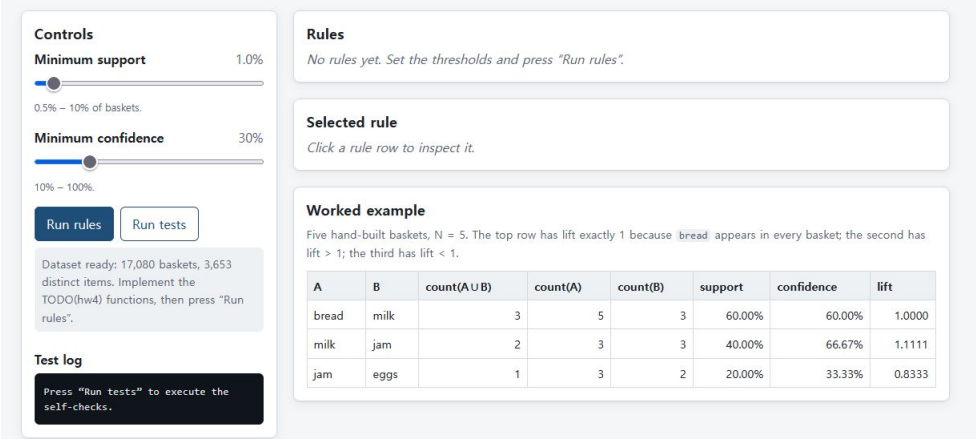
See tutorials	See models	See demos
Tutorials show you how to use TensorFlow.js with complete, end-to-end examples.	Pre-trained, out-of-the-box models for common use cases.	Live demos and examples run in your browser using TensorFlow.js.



[TensorFlow.js | Machine Learning for JavaScript Developers]

TensorFlow.js

HW4. Association Rules



1. Implementation Task

- **Repository:** [dryjins/RecSys-LLMs/tree/main/week4]
- **Dataset:** [UCI Online Retail dataset]. Treat each InvoiceNo as one transaction; use StockCode to identify items and Description to label them.
- **Basket Construction:** Create an invoice–item basket matrix.
- **Association Rule Mining:** Use Apriori or FP-Growth to identify frequent itemsets and generate association rules.
- **Rule Evaluation:** Report support, confidence, and lift for the selected rules.
- **Rule Filtering:** State and justify your minimum support and confidence thresholds; retain rules with **lift > 1** for further analysis.

2. Business & Algorithmic Analysis

- **(Mandatory) Useful vs. Misleading Rules:** Present one rule worth investigating and one rule to reject. Justify both using transaction counts, support, confidence, and lift.
- **(Below Optional)**
- **Rule Direction:** Compare **A** → **B** with **B** → **A**. Explain why their confidence values can differ.
- **Threshold Trade-off:** Explain how changing minimum support or confidence affects the number and usefulness of discovered rules.
- **Validation Before Action:** Check whether a selected rule remains stable in a later time period. Propose a controlled cross-sell, bundle, or product-placement experiment.
- **Association ≠ Causation:** Explain why high lift alone does not prove that an intervention will increase sales.

HW5. From Ratings to Recommendations

1. Implementation Task

https://dryjins.github.io/RecSys-LLMs/week5/

- Repository: dryjins/RecSys-LLMs/tree/main/week5
- Dataset: MovieLens 100K; use `week5/u.data` for user–movie ratings and `week5/u.item` for movie titles and genre indicators.1453
- Implementation: Build a JavaScript-based application to train and evaluate matrix factorization.
- Data Preparation: Report users, movies, ratings, and matrix sparsity. Treat missing ratings as unobserved, not zeros.
- Data Split: Sort each user's ratings by timestamp; use 80% for training, 10% for validation, and 10% for testing. Report exclusions of users or movies absent from training.
- Baseline: Predict ratings using the training-set mean.
- Matrix Factorization: Learn 16-dimensional user and movie vectors; predict ratings using their dot product.
- Training: Minimize squared error on observed training ratings. Select the checkpoint using validation RMSE; report baseline and MF test RMSE.
- Recommendations: Produce Top-10 lists for two test users, showing movie titles and scores. Exclude movies rated in training or validation.
- Evaluation: Treat test ratings ≥ 4.0 as relevant. Report average Recall@10 over users with eligible relevant test movies, using all training-set movies as candidates.

2. Business & Algorithmic Analysis

- **Familiar Choices vs. New Discoveries:** Do your recommendations help users discover movies, or mainly repeat their existing preferences? Compare two users' Top-10 lists with their past ratings. Use specific movies or genres to discuss familiar choices, new discoveries, and their value for a streaming platform.